RAJARATA UNIVERSITY OF SRI LANKA · DEPARTMENT OF COMPUTING

ICT 1306 — Object Oriented Programming · Group Assignment

FreshFold

Laundry & Dry Cleaning Management System

Installation, User & GitHub Manual

This manual explains how to

- install the required software (Visual Studio 2022, MySQL, Connector/NET, Git)
- create the MySQL database and connect the application to it
- build and run FreshFold in Visual Studio
- use every screen: login, dashboard, customers, orders, billing
- test the system and present the 5-minute demonstration
- upload the project to GitHub and work on it as a team

Team members

Name: _____ Index No.: _____

Name: _____ Index No.: _____

Name: _____ Index No.: _____

Name: _____ Index No.: _____

Repository: github.com/gamagemandira-ship-it/ICT1306-OOP-Project Manual version 1.0 · October 2026

Contents

1. Before you start 4

1.1 What you need 4

1.2 Overview of the setup 4

1.3 Technology used 5

2. Installing the required software 5

2.1 Visual Studio 2022 5

2.2 MySQL Server and MySQL Workbench 5

2.3 MySQL Connector/NET 6

2.4 Git for Windows 6

3. Getting the project files 6

3.1 From the ZIP file 6

3.2 From GitHub (after the project has been uploaded) 6

3.3 What is inside the project 7

4. Creating the MySQL database 7

4.1 Run the setup script in MySQL Workbench 7

4.2 Command-line alternative 8

4.3 Create a MySQL account for the application (recommended) 8

4.4 The database tables 8

5. Connecting the application to MySQL 8

6. Adding MySQL Connector/NET to the project 9

6.1 How the project uses the connector 9

7. Building and running in Visual Studio 10

8. First run: creating the administrator account 10

8.1 Changing or resetting the login 10

8.2 How to know the connection works 11

9. Using FreshFold 11

9.1 Dashboard 11

9.2 Customer Management 11

9.3 Order Management 12

9.4 Billing & Payment 13

9.5 Five-minute demonstration script 13

10. Connecting to Git and GitHub 13

10.1 Important words 14

10.2 One-time setup on every computer 14

10.3 First upload — Case A: the repository is empty 14

10.4 First upload — Case B: the repository already has files 15

10.5 Everyday team workflow 15

10.6 Working with branches (recommended for groups) 15

10.7 Solving a merge conflict 16

10.8 Using Git inside Visual Studio (alternative) 16

10.9 What must never be uploaded 16

11. Testing the system 17

11.1 Automated tests for the business classes 17

11.2 Manual tests 17

11.3 What has already been verified 17

12. Troubleshooting 18

12.1 Build problems 18

12.2 Problems when starting or using the program 18

12.3 Git and GitHub problems 19

13. OOP concepts — quick reference for the viva 19

Appendix A — Command summary 20

Appendix B — Setup checklist 21

1. Before you start

FreshFold is a desktop application for a small laundry and dry-cleaning shop. A staff member logs in, registers customers, creates orders, follows each order through the cleaning process, prints invoices and records payments. All data is stored in a real MySQL database.

Follow the chapters in order the first time. Every step says exactly what to click or type. Text in `this font` is something you type or a file name.

1.1 What you need

Item	Details
Computer	Windows 10 or Windows 11, 64-bit, about 15 GB free disk space, internet connection for installing
Project files	The `FreshFold.zip` file, or access to the GitHub repository
Visual Studio 2022	Community edition is free. Needs the C++ workload and the C++/CLI component (chapter 2)
MySQL Server 8.0+	With MySQL Workbench. Installed with the MySQL Installer
MySQL Connector/NET	The official MySQL driver used by the program to talk to MySQL
Git for Windows	Only needed for uploading to and downloading from GitHub (chapter 10)
GitHub account	Each team member needs one; the repository owner must give access

1.2 Overview of the setup

Step	What you do	Chapter
1	Install Visual Studio, MySQL, Connector/NET and Git	2
2	Unzip the project (or clone it from GitHub)	3
3	Create the `freshfold_db` database by running the SQL script	4
4	Create the `config\database.ini` file with your MySQL user name and password	5
5	Copy the Connector/NET DLL files into `lib\mysql`	6
6	Open the solution, build and run	7
7	Create the first administrator account and log in	8

1.3 Technology used

Part	Technology
Programming language	C++ (standard C++14 for the business classes; C++/CLI for the screens and database access)
User interface	Windows Forms (.NET Framework 4.8) through C++/CLI
Database	MySQL 8.0.16 or newer
Database connector	MySQL Connector/NET (`MySql.Data.dll`)
Password security	PBKDF2-HMAC-SHA256 with 100,000 iterations and a random salt; only the hash is stored
Version control	Git and GitHub

> **Note:** A Windows Forms program written in C++ must be compiled as C++/CLI (the `/clr` option). MySQL's native C++ connector does not fit well into such a project, so FreshFold uses MySQL's official .NET connector, Connector/NET, which works directly from C++/CLI. The program itself is still written entirely in C++.

2. Installing the required software

2.1 Visual Studio 2022

1. Download **Visual Studio 2022 Community** from `https://visualstudio.microsoft.com/downloads/` and run the installer.
2. On the **Workloads** tab, tick **Desktop development with C++**.
3. Open the **Individual components** tab and search for and tick:
   • **C++/CLI support for v143 build tools (Latest)**
   • **.NET Framework 4.8 targeting pack**
4. Click **Install** and wait until it finishes. Restart the computer if asked.

> **Tip:** Already have Visual Studio 2022? Open the **Visual Studio Installer**, click **Modify** on your installation and add the two components above.

2.2 MySQL Server and MySQL Workbench

1. Download the **MySQL Installer for Windows** from `https://dev.mysql.com/downloads/installer/`.
2. Run it and choose a setup type that includes **MySQL Server** and **MySQL Workbench** (for example **Custom**, then add both products).
3. During configuration keep the default port **3306**.
4. Set a **root password** and write it down — you need it to create the database.
5. Keep **Configure MySQL Server as a Windows Service** ticked (service name usually **MySQL80**) and **Start the MySQL Server at System Startup**.
6. Finish the installer. Open **MySQL Workbench**, click the **Local instance** connection and enter the root password to check that MySQL is running.

2.3 MySQL Connector/NET

1. Download **Connector/NET** (Windows installer) from `https://dev.mysql.com/downloads/connector/net/`.
2. Run the installer with the default options.
3. Remember the installation folder. By default it is under `C:\Program Files (x86)\MySQL\` in a folder named like `MySQL Connector NET <version>`. You need it in chapter 6.

2.4 Git for Windows

1. Download Git from `https://git-scm.com/download/win` and run the installer.
2. The default options are fine. Git for Windows includes **Git Credential Manager**, which lets you sign in to GitHub through your web browser.
3. Open **Command Prompt** and check the installation:

```
git --version
```

3. Getting the project files

3.1 From the ZIP file

1. Right-click `FreshFold.zip` → **Extract All…**
2. Choose a short path without special characters, for example `C:\Projects`. You will get `C:\Projects\FreshFold`.

> **Important:** Do not run the project from inside the ZIP file or from a OneDrive-synchronised folder. Extract it to a normal folder first.

3.2 From GitHub (after the project has been uploaded)

If a team member has already uploaded the project (chapter 10), download it with:

```
cd C:\Projects
git clone https://github.com/gamagemandira-ship-it/ICT1306-OOP-Project.git
```

Everything in this manual then happens inside the cloned `ICT1306-OOP-Project` folder.

3.3 What is inside the project

Folder / file	Contents
`FreshFold.sln`	Visual Studio solution — double-click to open the project
`FreshFold.vcxproj`	C++/CLI project settings (.NET Framework 4.8, x64)
`src\Models`	Business classes: Customer, LaundryOrder, DryCleaningOrder, OrderFactory, PriceList
`src\Database`	DatabaseManager (all SQL) and DatabaseConfig (reads the settings file)
`src\Services`	AuthenticationService (login) and BillingService (bills, invoices, payments)
`src\Forms`	The screens: Login, Dashboard, Customers, Orders, Billing
`database`	`freshfold_db.sql` (creates the database) and `sample_data.sql` (optional demo data)
`config`	`database.example.ini` — template for your own connection settings
`lib\mysql`	Place for the Connector/NET DLL files (not uploaded to GitHub)
`docs`	Class diagram, OOP explanations and the testing checklist
`tests`	`ModelTests.cpp` — console tests for the business classes
`screenshots`	Put your own screenshots here for the report and README

4. Creating the MySQL database

4.1 Run the setup script in MySQL Workbench

1. Open **MySQL Workbench** and connect to **Local instance** as **root**.
2. Click **File → Open SQL Script…** and open `database\freshfold_db.sql` from the project folder.
3. Click the **Execute** button (the lightning icon ⚡) above the script.
4. In the **Navigator** panel on the left, click the refresh icon next to **SCHEMAS**. You should see `freshfold_db` with four tables: `customers`, `orders`, `payments` and `users`.

Optional demo data: open and execute `database\sample_data.sql` the same way. It adds three invented customers and three orders, which is useful for the demonstration.

> **Note:** Both scripts are **safe to run again**. They never delete data and do not create duplicates. To start again with an empty database, run `DROP DATABASE freshfold_db;` and then run the scripts again (this deletes all data).

4.2 Command-line alternative

```
cd C:\Projects\FreshFold
mysql -u root -p < database\freshfold_db.sql
mysql -u root -p < database\sample_data.sql
```

If `mysql` is not recognised, use the full path, for example `"C:\Program Files\MySQL\MySQL Server 8.0\bin\mysql.exe"`.

4.3 Create a MySQL account for the application (recommended)

It is better not to use the root account in the application. In Workbench, open a new query tab (**File → New Query Tab**), paste the following, replace the password with your own, and execute it once:

```
CREATE USER IF NOT EXISTS 'freshfold_app'@'localhost'
    IDENTIFIED BY 'ChooseAStrongPassword123!';
GRANT SELECT, INSERT, UPDATE, DELETE ON freshfold_db.* TO 'freshfold_app'@'localhost';
FLUSH PRIVILEGES;
```

The same three lines are at the end of `freshfold_db.sql` as comments.

4.4 The database tables

Table	Purpose	Important rules
`users`	Staff login accounts	Username is unique; only a password hash is stored
`customers`	Customer details	Name, phone and address are required; phone number is unique
`orders`	Laundry and dry-cleaning orders	Linked to a customer; quantity > 0; status and payment status limited to allowed values; money stored as DECIMAL
`payments`	Cash/Card payments	Linked to an order; amount > 0; method is Cash or Card

5. Connecting the application to MySQL

The program reads its MySQL connection settings from a small text file, `config\database.ini`. This file contains your password, so it is **never uploaded to GitHub** (it is listed in `.gitignore`). Every team member creates their own copy.

1. Open the `config` folder in the project.
2. Copy `database.example.ini` and rename the copy to `database.ini`.
3. Open `database.ini` with Notepad and enter your values:

```
[mysql]
server=localhost
port=3306
database=freshfold_db
user=freshfold_app
password=ChooseAStrongPassword123!
```

4. Use `user=root` and your root password if you skipped section 4.3.

5. Save the file. Make sure Windows did not save it as `database.ini.txt` (in File Explorer tick **View → File name extensions** to check).

> **Tip:** The program finds the file automatically: it looks next to `FreshFold.exe` and in a `config` folder in the exe folder or up to four parent folders. When you run from Visual Studio it finds `<project>\config\database.ini` without any extra setting.

6. Adding MySQL Connector/NET to the project

1. Open the Connector/NET installation folder (section 2.3), then its **Assemblies** folder.
2. Open the folder for **.NET Framework 4.8** (if there is no 4.8 folder, use the closest .NET Framework 4.x folder).
3. Select **all** `.dll` files in that folder and copy them.
4. Paste them into the project's `lib\mysql` folder (it already contains a `README.txt`).

> **Important:** Copy **all** DLL files, not only `MySql.Data.dll`. MySql.Data needs its helper libraries (for example Google.Protobuf, BouncyCastle, K4os compression and ZstdSharp) when the program runs.

6.1 How the project uses the connector

Everything is already configured in `FreshFold.vcxproj`; you do not need to change project settings.

Setting	Value
Connector	MySQL Connector/NET — .NET assembly `MySql.Data.dll`
Reference	`MySql.Data` with hint path `$(MySqlConnectorDir)MySql.Data.dll`, Copy Local = true
MySqlConnectorDir	`lib\mysql\` inside the project folder
Include / library directories	Not needed — Connector/NET is a .NET assembly, not a native .lib
Linker dependencies	Not needed
Runtime DLLs	A post-build step copies every DLL from `lib\mysql` next to `FreshFold.exe`

To check: after opening the solution (chapter 7), expand **FreshFold → References** in Solution Explorer. `MySql.Data` must be listed without a yellow warning triangle.

7. Building and running in Visual Studio

1. Double-click `FreshFold.sln` (or in Visual Studio: **File → Open → Project/Solution**).
2. In the toolbar, choose configuration **Debug** and platform **x64**.
3. Click **Build → Build Solution** (Ctrl+Shift+B). Watch the **Output** window; it should end with `Build: 1 succeeded`.
4. Click **Debug → Start Debugging** (F5). The FreshFold login window appears.

The program file is created at `x64\Debug\FreshFold.exe` inside the project folder. You can also start that file directly, as long as the DLLs were copied next to it (the build does this automatically).

> **Note:** The screens are written in code, not with the Forms Designer. If you double-click a form file and an empty designer opens, close it and use **right-click → View Code**.

> **Important:** The C++/CLI part of the project (screens, database layer, login) could not be compiled in the environment where the project was generated, because no Windows/Visual Studio was available. If the first build shows compiler errors, copy the exact error messages so they can be fixed. See chapter 12 for common build problems.

8. First run: creating the administrator account

FreshFold has **no built-in default password**. The first time it starts with an empty `users` table, the login window switches to **First-Time Setup** mode.

1. Enter a username: 3–50 characters, letters, digits, `.`, `_` or `-` (for example `admin`).
2. Enter a password of at least 8 characters in **Password** and again in **Confirm password**.
3. Click **Create Administrator**. A message confirms that the account was created.
4. The window switches to normal **Staff Login**. Enter the same username and password and click **Login**.
5. The message *Login successful. Welcome, admin!* appears and the Dashboard opens.

8.1 Changing or resetting the login

To remove the account (for example to set a new password before the demonstration), run this in Workbench and restart FreshFold. The First-Time Setup screen appears again:

```
USE freshfold_db;
DELETE FROM users;
```

8.2 How to know the connection works

If the login window shows **Staff Login** or **First-Time Setup** without a red error message, the program has connected to MySQL. If it shows a red message and a **Retry Connection** button, read the message and check chapter 12.

> **Note:** After 5 incorrect login attempts the Login button is disabled for 30 seconds. Unknown usernames and wrong passwords give the same message, so the screen does not reveal which usernames exist.

9. Using FreshFold

9.1 Dashboard

The dashboard shows four live numbers loaded from MySQL: **Total Customers**, **Total Orders** (cancelled orders are not counted), **Pending Orders** and **Ready for Pickup**. The menu on the left opens the other screens. Each time you close a screen, the numbers are refreshed. If MySQL is not available, the cards show `--` and an error message — the program never shows invented numbers.

Menu button	Action
Dashboard	Reloads the statistics
Customers	Opens Customer Management
Orders	Opens Order Management
Billing	Opens Billing & Payment
Logout	Asks for confirmation, then returns to the login window

9.2 Customer Management

Task	How to do it
Add a customer	Fill in **Name**, **Contact Number** and **Address** (Customer ID stays empty) → **Add Customer**. The new ID appears in the Customer ID box.
View all customers	Click **View Customers**. The newest customers are listed first.
Search	Type an ID, part of a name or part of a phone number → **Search** (or press Enter).
Edit	Click a row in the table (the details load into the form) → change the values → **Update Customer**.
Delete	Click a row → **Delete Customer** → confirm with **Yes**. Customers who have orders cannot be deleted.
Start again	**Clear Form** empties the form so you can add a new customer.

Validation: name, contact number and address are required. The contact number may contain digits, spaces, `+ - ( )` and needs at least 7 digits. Two customers cannot have the same contact number.

9.3 Order Management

Prices (set in `src\Models\PriceList.h`):

Service	Charged by	Price
Wash & Fold	Weight in kg (decimals allowed, e.g. 2.5)	Rs. 350.00 per kg
Steam Press	Number of items (whole number)	Rs. 120.00 per item
Dry Cleaning	Number of items (whole number)	Rs. 600.00 per item + Rs. 150.00 handling fee per order

Task	How to do it
Calculate a bill	Choose a **Service** (the quantity label changes to *Weight (kg)* or *Number of Items*), enter the quantity → **Calculate Bill**. The calculation is shown in the bill box.
Create an order	Choose the **Customer**, service, quantity and **Order Date** → **Create Order**. New orders start as *Pending* and *Unpaid*.
Load an order	Click its row in the table.
Change details	Load the order, change customer/service/quantity/date → **Update Order**. Only possible while the order is *Pending* and *Unpaid*.
Move status forward	Load the order. The **Processing Status** box already shows the next status → **Update Status**.
Cancel	Load the order → **Cancel Order** → **Yes**. The order stays in the list with status *Cancelled*.
Bill & payment	Load the order → **Bill & Payment** opens the billing screen for that order.
Search	Search by order ID, customer name, phone, service, status or payment status.

Order status workflow

```
Pending  -->  In Wash  -->  Ready for Pickup  -->  Completed
   |              |                |
   +--------------+----------------+-->  Cancelled  (only if nothing has been paid)
```

- The status moves **one step at a time**; steps cannot be skipped or reversed.
- An order can only be marked **Completed** when it is fully **Paid**.
- Completed orders and orders with a payment cannot be cancelled.

9.4 Billing & Payment

1. Enter the **Order ID** → **Load Order** (or open the screen from the Orders screen).
2. Check the summary (customer, service, quantity, total, paid so far, balance due, payment status) and the invoice on the right.
3. Choose the **Payment method** (Cash or Card). The **Amount** box already contains the balance due; change it for a partial payment.
4. Click **Record Payment** and confirm. The payment status becomes *Partially Paid* or *Paid* and the payment appears in **Payment History**.
5. Click **Print Invoice** to open a print preview. Print it, or choose **Microsoft Print to PDF** to save it as a PDF.

Rules: the amount must be greater than zero, have at most 2 decimal places, and cannot be more than the balance due. The payment and the new payment status are saved together in one database transaction — if anything fails, nothing is saved.

9.5 Five-minute demonstration script

Time	Show this	Say this (OOP point)
0:00	Log in; show the dashboard numbers	Login uses AuthenticationService; passwords are hashed (PBKDF2)
0:45	Customers: add a customer, search, edit, update	A **Customer object** is created; private data + validating setters = **encapsulation**
1:45	Try to add a customer with an empty name / phone abc	The error message comes from the Customer setters
2:15	Orders: Steam Press 3 items → Calculate Bill (Rs. 360.00); Dry Cleaning 3 items → Calculate Bill (Rs. 1950.00)	Same **calculateBill()** call, different result: **polymorphism**; DryCleaningOrder **inherits** LaundryOrder; the screen does not know the formula: **abstraction**
3:15	Create the Dry Cleaning order; Update Status to In Wash, then Ready for Pickup	Status rules are inside the LaundryOrder class
3:45	Bill & Payment: pay Rs. 500 Cash, then the balance by Card; Print Invoice	BillingService + database transaction
4:30	Back in Orders: Update Status to Completed; return to the dashboard	Numbers changed — they come from MySQL

10. Connecting to Git and GitHub

Git keeps the history of every change to the project. GitHub stores the project online so that all team members can work on the same code. The project repository is:

```
https://github.com/gamagemandira-ship-it/ICT1306-OOP-Project
```

10.1 Important words

Word	Meaning
Repository (repo)	The project folder together with its full change history
Clone	Download a copy of a GitHub repository to your computer
Commit	A saved snapshot of your changes, with a short message
Push	Upload your commits to GitHub
Pull	Download your team members' new commits from GitHub
Branch	A separate line of work, so you can change code without disturbing the main version
Pull request	A request on GitHub to merge a branch into `main`, so others can review it first
`.gitignore`	List of files Git must never upload (passwords, build output, DLLs)

10.2 One-time setup on every computer

1. Install Git (section 2.4) and create a free account at `https://github.com`.
2. Tell Git your name and e-mail (use the e-mail of your GitHub account). Open Command Prompt and type:

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
git config --global init.defaultBranch main
```

3. **Get access to the repository.** The repository owner opens the repository on GitHub → **Settings** → **Collaborators** → **Add people**, and adds each team member's GitHub username. Each member accepts the invitation from their e-mail or GitHub notifications.
4. **Sign in.** The first time you push, Git Credential Manager opens a GitHub sign-in window in your browser. Sign in and allow access. Git remembers this, so you only do it once per computer.

> **Note:** GitHub does not accept your account password on the command line. Use the browser sign-in from Git Credential Manager (recommended) or a **personal access token** (GitHub → Settings → Developer settings → Personal access tokens) in place of the password.

10.3 First upload — Case A: the repository is empty

Use this if the GitHub page of the repository shows *Quick setup* and no files. Only **one** team member does this.

```
cd C:\Projects\FreshFold
git init
git status
git add .
git commit -m "Add FreshFold laundry management system"
git branch -M main
git remote add origin https://github.com/gamagemandira-ship-it/ICT1306-OOP-Project.git
git push -u origin main
```

> **Important:** Before `git add .` look at the output of `git status`. The following must **not** be listed: `config/database.ini`, DLL files in `lib/mysql`, the `x64` folder and the `.vs` folder. They are excluded by `.gitignore` — if you see them, stop and check that the `.gitignore` file is in the project folder.

10.4 First upload — Case B: the repository already has files

Use this if the repository already contains files (for example a README). Do **not** force-push — that would delete your team's work. Clone the repository first and copy the project into it:

```
cd C:\Projects
git clone https://github.com/gamagemandira-ship-it/ICT1306-OOP-Project.git
REM Copy everything from C:\Projects\FreshFold into
REM C:\Projects\ICT1306-OOP-Project with File Explorer.
REM If a file with the same name exists (e.g. README.md), compare them first.
cd ICT1306-OOP-Project
git status
git add .
git commit -m "Add FreshFold laundry management system"
git push
```

If the existing README has useful content, merge it into the FreshFold README instead of simply replacing it.

10.5 Everyday team workflow

Every time you work on the project, follow this cycle. Pull first, so you start from the latest version:

```
git pull                          (1) get your team's latest changes
REM ... edit and test the code in Visual Studio ...
git status                        (2) see which files you changed
git add src/Forms/OrderForm.cpp   (3) choose the files to save (or: git add .)
git commit -m "Improve order search message"   (4) save a snapshot
git push                          (5) upload to GitHub
```

- Write short, clear commit messages that say **what** changed (for example “Add quantity validation to order form”).
- Commit small, working steps. Build the project before you commit.
- Pull often, and push your work at the end of every session.

10.6 Working with branches (recommended for groups)

Branches let team members work at the same time without breaking the main version:

```
git pull
git checkout -b feature/billing-print        create and switch to a new branch
REM ... change, build, test ...
git add .
git commit -m "Add print preview to billing screen"
git push -u origin feature/billing-print     upload the branch
```

1. On GitHub, a yellow bar offers **Compare & pull request**. Click it, describe the change and create the pull request.

2. A team member reviews it and clicks **Merge pull request**.
3. Everyone then updates their `main` branch:

```
git checkout main
git pull
```

10.7 Solving a merge conflict

A conflict happens when two people change the same lines. Git marks the file like this:

```
<<<<<<< HEAD
lblStatus->Text = "Order saved.";
=======
lblStatus->Text = "Order created successfully.";
>>>>>>> origin/main
```

1. Open the file in Visual Studio, keep the correct version and delete the marker lines `<<<<<<<`, `=======` and `>>>>>>>`.
2. Build the project to make sure it still compiles.
3. Run `git add <file>`, then `git commit -m "Resolve merge conflict"` and `git push`.

10.8 Using Git inside Visual Studio (alternative)

- **Clone:** Git → Clone Repository… → paste the repository URL → Clone.
- **See changes and commit:** View → Git Changes → type a message → **Commit All**.
- **Push / Pull:** the arrow buttons at the top of the Git Changes window.
- **Branches:** the branch name in the bottom-right status bar → New Branch.

10.9 What must never be uploaded

Never upload	Why	Protected by
`config/database.ini`	Contains your MySQL password	`.gitignore`
DLLs in `lib/mysql`	Third-party binaries; each member copies them	`.gitignore`
`x64/`, `Debug/`, `Release/`, `*.exe`	Generated by the build	`.gitignore`
`.vs/`, `*.user`	Personal Visual Studio settings	`.gitignore`
Real customer data, API keys, passwords	Privacy and security	You — check `git status` before committing

> **Important:** If a password was committed by mistake, deleting the file in a new commit is not enough — it is still in the history. Change that MySQL password immediately and ask your lecturer or a team member experienced with Git for help cleaning the history.

11. Testing the system

11.1 Automated tests for the business classes

`tests\ModelTests.cpp` checks validation, billing, the status workflow, runtime polymorphism and the configuration file reader (52 checks). It does not need MySQL. Open **x64 Native Tools Command Prompt for VS 2022** from the Start menu and run:

```
cd C:\Projects\FreshFold
cl /EHsc /std:c++14 /I src tests\ModelTests.cpp ^
   src\Models\Customer.cpp src\Models\LaundryOrder.cpp ^
   src\Models\DryCleaningOrder.cpp src\Models\OrderFactory.cpp ^
   src\Database\DatabaseConfig.cpp /Fe:ModelTests.exe
ModelTests.exe
```

The last line of the output should be `52 passed, 0 failed`.

11.2 Manual tests

Open `docs\testing-checklist.md`. It contains 33 test cases (login, customers, orders, bills, payments, a stopped MySQL server, invalid input, SQL-injection attempts and more). For each test, follow the steps, write what actually happened in the **Actual result** column and mark Pass or Fail. Only mark a test as passed after you have really run it.

11.3 What has already been verified

Part	Status
Business classes and ModelTests.cpp	Compiled with g++ and clang (C++14, no warnings); all 52 checks pass
BillingService logic	Compiled and tested with a stand-in for the database class
SQL scripts	Executed on MySQL 8.0.46, twice, to confirm they are safe to re-run
SQL statements in DatabaseManager.cpp	The exact SQL text was run on MySQL 8.0.46 with parameters (40 checks)
C++/CLI code and project file	**Not yet compiled** — build in Visual Studio 2022 and report any errors
Running application, connector, printing	**Not yet run** — use the manual testing checklist

12. Troubleshooting

12.1 Build problems

Problem	Solution
Error that `MySql` / `MySql::Data` cannot be found, or a yellow warning on the MySql.Data reference	The DLLs are missing from `lib\mysql`. Repeat chapter 6, then rebuild.
Error that C++/CLI or `/clr` is not supported	Install **C++/CLI support for v143 build tools** in the Visual Studio Installer.
Error about .NET Framework 4.8 or reference assemblies	Install the **.NET Framework 4.8 targeting pack**.
Platform toolset v143 not found	Use Visual Studio 2022, or right-click the project → Properties → General → Platform Toolset and choose your installed version.
Other compiler errors	Copy the first error from the **Error List** (with file name and line number) so it can be fixed.

12.2 Problems when starting or using the program

Message / problem	Solution
The database configuration file was not found	Create `config\database.ini` (chapter 5).
Cannot connect to the MySQL server	Start MySQL: Windows key → `services.msc` → **MySQL80** → Start. Check server and port in database.ini.
MySQL rejected the user name or password	Check user and password in database.ini, or create the user (section 4.3).
The database named in database.ini was not found	Run `database\freshfold_db.sql` (chapter 4).
A required table is missing	Run `database\freshfold_db.sql` again; it only adds what is missing.
FileNotFoundException for MySql.Data or another DLL	Copy **all** DLLs from the Connector/NET .NET Framework 4.8 folder to `lib\mysql` and rebuild.
Retrieval of the RSA public key is not enabled	SSL is turned off on the MySQL server. Turn SSL back on (default in MySQL 8) or ask for help changing the user's authentication plugin.
Print Invoice shows an error	Install a printer, or make sure **Microsoft Print to PDF** is enabled (Windows Features).
Need technical details of an error	Run with F5 and open the **Output** window — full database errors are written there, not shown to users.

12.3 Git and GitHub problems

Message	Meaning and solution
`fatal: not a git repository`	You are in the wrong folder. Use `cd` to go into the project folder.
`error: src refspec main does not match any`	Nothing has been committed yet. Run `git add .` and `git commit` first.
`error: remote origin already exists`	The remote is already set. Check it with `git remote -v`; change it with `git remote set-url origin <URL>`.
`! [rejected] ... (fetch first)` or `non-fast-forward`	GitHub has commits you do not have. Run `git pull`, fix any conflicts, then `git push`. Do not use `--force`.
`Permission denied` / `403`	Your account has no access. Ask the owner to add you as a collaborator and accept the invitation; then sign in again.
`Authentication failed`	Sign in through the browser window, or use a personal access token instead of your password.
`refusing to merge unrelated histories`	Happens when `git init` was used although the repository already had files. Use Case B (section 10.4) instead.
database.ini appears in `git status`	The `.gitignore` file is missing or the file was added earlier. Run `git rm --cached config/database.ini` and commit.

13. OOP concepts — quick reference for the viva

Detailed explanations with code are in `docs\oop-examples.md`; the class diagram is in `docs\class-diagram.md`.

Concept	Where in the code	Short explanation
Class	`Customer`, `LaundryOrder` in `src\Models`	A blueprint that groups data and the methods that work on it.
Object	`CustomerForm::onAddClick`, `DatabaseManager::findCustomerById`, `createOrder`	A real instance created at runtime, e.g. when a customer is registered or loaded from MySQL.
Encapsulation	Private members; validating setters such as `setPhone`, `setQuantity`	Data can only change through methods that check the value. There is no setter for the total.
Abstraction	`calculateBill()`, `BillingService::quote()`, `PriceList.h`	Screens ask for a bill without knowing the formula; all prices are in one file.
Inheritance	`class DryCleaningOrder : public LaundryOrder`	Dry cleaning reuses everything from a normal order and adds a handling fee.
Polymorphism	`virtual double calculateBill() const` overridden in DryCleaningOrder; called through `LaundryOrder&` and `unique_ptr<LaundryOrder>`	The same call runs the right version for the real object type at runtime.

Appendix A — Command summary

MySQL

```
mysql -u root -p < database\freshfold_db.sql      create database and tables
mysql -u root -p < database\sample_data.sql      optional demo data
DROP DATABASE freshfold_db;                      reset (deletes all data)
DELETE FROM users;                               show first-time setup again
```

Git — first time

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
git clone https://github.com/gamagemandira-ship-it/ICT1306-OOP-Project.git
```

Git — every day

```
git pull
git status
git add .
git commit -m "Describe the change"
git push
```

Git — branches

```
git checkout -b feature/my-change
git push -u origin feature/my-change
git checkout main
git pull
```

Appendix B — Setup checklist

Done	Step
☐	Visual Studio 2022 with Desktop C++, C++/CLI v143 and .NET Framework 4.8 targeting pack
☐	MySQL Server + Workbench installed, service MySQL80 running
☐	Connector/NET installed
☐	Git installed and `user.name` / `user.email` configured
☐	Project extracted or cloned to `C:\Projects`
☐	`freshfold_db.sql` executed (and optionally `sample_data.sql`)
☐	Application MySQL user created
☐	`config\database.ini` created with your own password
☐	All Connector/NET DLLs copied to `lib\mysql`
☐	Solution builds in Debug | x64
☐	Administrator account created and login works
☐	Testing checklist completed
☐	Project pushed to GitHub; `git status` confirmed no password or DLL was uploaded
☐	Screenshots added to `screenshots` and team members added to README